Consolidated Technical Report: QML Workspace

Repository: qml

March 13, 2026

Abstract

This report provides a maintainer-oriented technical synthesis of the multi-task QML workspace. The focus is not only on what was implemented, but on why specific design choices were made, what trade-offs they encode, and where the code should evolve next. The repository combines foundational quantum-circuit validation, graph neural network baselines for jet tagging, technical positioning on QML practice, and a contrastive quantum representation learning pipeline for MNIST. All sections are written to support future contributors who need to reproduce, modify, and extend the codebase with minimal ambiguity.

1 Purpose and maintainer perspective

This document is written as a **maintainer-facing rationale** for the codebase. It focuses on why each implementation choice was made, which assumptions are encoded, and what trade-offs are accepted in the current version.

The repository is intentionally task-partitioned:

- `task1`: quantum-circuit primitives and correctness checks,
- `task2`: classical GNN baselines for HEP jet tagging,
- `task3`: technical position statement and research direction,
- `task6`: quantum contrastive representation learning.

This partitioning prevents cross-task coupling and keeps each deliverable independently reviewable.

2 Methodological framing

Across tasks, the implementation strategy follows three principles:

1. **Operational traceability**: each experiment leaves concrete artifacts (figures, logs, checkpoints) in stable paths.
2. **Progressive complexity**: start from verifiable primitives (Task 1) before layered training systems (Task 2, Task 6).
3. **Failure-informed hardening**: when numerical or execution failures appear, encode fixes directly in code (not only in notes).

3 Task 1 rationale: establish quantum baseline primitives

3.1 Why this task exists

Task 1 serves as a correctness baseline for two essential operations reused conceptually later: state preparation and state-comparison via SWAP test. Before introducing trainable quantum models, we validate deterministic gate compositions and measurement semantics.

3.2 Why these gates and layout were chosen

The five-qubit chain (Hadamards, nearest-neighbor CNOTs, boundary SWAP, and $R_X(\pi/2)$) is deliberately minimal but expressive enough to exercise:

- superposition creation,
- local entanglement propagation,
- non-local wire permutation,
- parameterized rotation behavior.

The second circuit uses SWAP test because it produces a scalar overlap proxy from one ancilla measurement stream, which is the same operational principle used in Task 6.

3.3 Maintainability decisions

Both scripts produce both graphical and text-diagram outputs. The text form is retained because it is diff-friendly in logs and CI-like environments, while PNG diagrams are reviewer-friendly.

4 Task 2 rationale: physically motivated graph induction + model diversity

4.1 Why graph representation for jets

Jets are variable-cardinality particle sets with meaningful local geometry in $(\Delta\eta, \Delta\phi)$. A graph encoding captures relational structure that flat tabular representations miss.

4.2 Why the specific feature construction

Node features include relative angular coordinates and log-scaled kinematics. This was chosen to:

- encode geometry relative to jet axis (translation-invariant in detector coordinates),
- compress heavy-tailed momentum/energy scales,
- include relative fractions to reduce absolute-scale dependence.

4.3 Why two architectures (DGCNN and GAT)

The pair was selected to isolate a core inductive-bias choice:

- **DGCNN**: dynamic neighborhood recomputation in learned space (higher flexibility),
- **GAT**: fixed neighborhood with learned edge weighting (higher interpretability).

This is more informative for maintainers than training two near-identical models.

4.4 Engineering notes that matter for maintenance

- Output directories are explicit (`figures/`, `models/`, `logs/`) to avoid clutter.
- Windows console encoding issues were mitigated by avoiding Unicode arrows in runtime prints.
- Dataset loader was hardened to handle ParticleNet shard schema variations.

From `task2/logs/smoke_run.log`, current smoke comparison metrics are:

Table 1: Task 2 test metrics (smoke compare run)

Model	Accuracy	ROC AUC
DGCNN	0.5167	0.8038
GAT	0.5533	0.7883

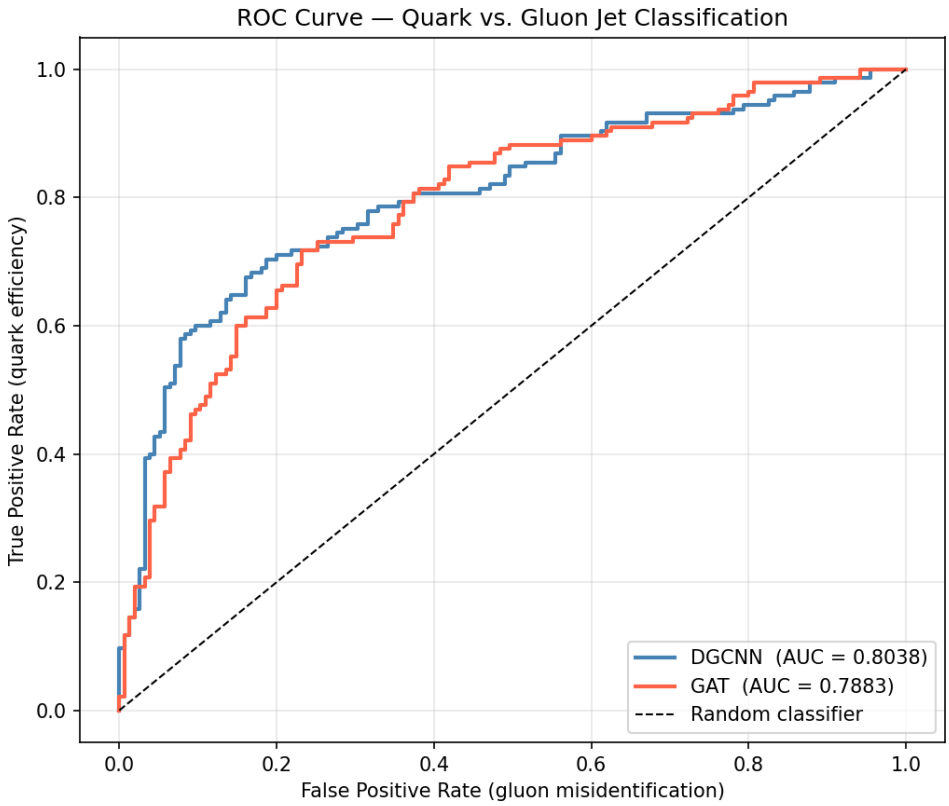

Figure 1: Task 2 ROC comparison (DGCNN vs GAT).

Reading these metrics correctly. The smoke-run numbers are intended as a reproducibility sanity check rather than a final benchmark. Their value to maintainers is that they confirm end-to-end health (data pipeline, model training, evaluation, figure generation) after structural changes.

5 Task 3 rationale: document decision intent, not only implementation

Task 3 is not executable code, but it is retained as a maintainability artifact. It captures explicit technical positioning on QML realism (NISQ constraints, benchmarking rigor, algorithm/tool preferences), which informs future implementation priorities and acceptance criteria.

6 Task 6 rationale: contrastive quantum representation learning under NISQ constraints

6.1 Why this architecture

Task 6 uses a small classical pre-network followed by amplitude embedding and trainable variational layers, then evaluates pair similarity with a SWAP test.

The design objective is to decouple responsibilities:

- classical front-end performs coarse compression from image space,
- quantum circuit learns representation geometry under shared trainable parameters,
- SWAP test provides a fidelity-based similarity scalar for contrastive supervision.

6.2 Why this contrastive loss

The selected objective

$$\mathcal{L} = y(1 - F)^2 + (1 - y)F^2 \tag{1}$$

is intentionally simple, bounded, and directly aligned with fidelity semantics. It avoids margin hyperparameters in the initial implementation and keeps debugging surface small.

6.3 Stability interventions and rationale

Longer runs exposed numerical instability (invalid state norms / NaNs). The maintained code now includes:

- NaN/Inf sanitization on encoded vectors and trainable parameters,
- bounded activation (`tanh`) before normalization,
- normalization with explicit epsilon,
- gradient clipping,
- subset-aware class sampling to avoid missing-class key errors on small splits.

These are not cosmetic: they are required for robust execution in constrained local environments.

6.4 Current measurable results

Validated run (`task6/outputs`, 3 epochs):

- Best epoch: 2 / 3,
- Best validation loss: 0.1451,
- Best validation same-class fidelity: 0.7556,

- Best validation different-class fidelity: 0.2593.

  Stronger run checkpoint (`task6/outputs_strong`, 4 epochs):

- Best validation loss: 0.1258,
- Best validation same-class fidelity: 0.6906,
- Best validation different-class fidelity: 0.2324.

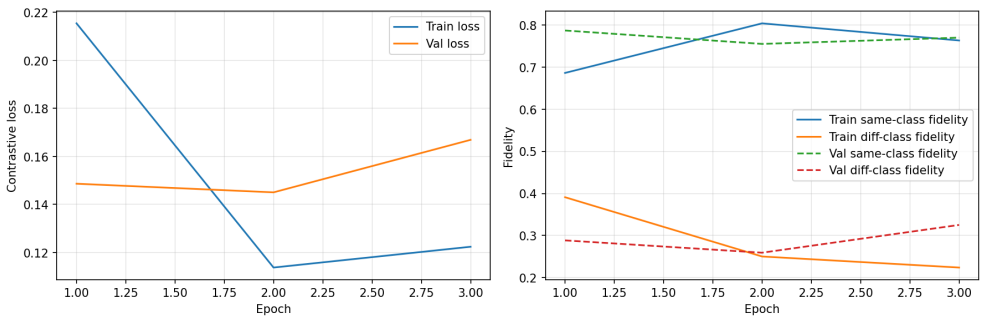

Figure 2: Task 6 training curves (validated extended run).

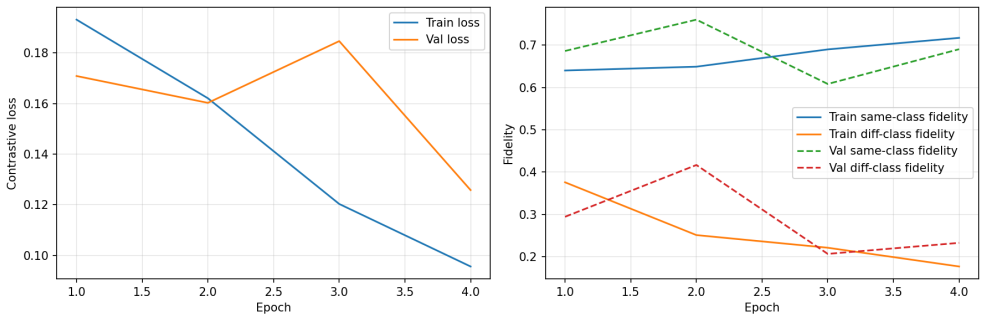

Figure 3: Task 6 training curves (stronger run).

Interpretation for future work. The persistent gap between same-class and different-class fidelity indicates that the representation is learning class-relevant geometry, but variance across runs suggests optimization sensitivity remains nontrivial. In practice, this motivates seed-sweep reporting and possibly a scheduler-based learning-rate policy in future revisions.

7 Repository publication readiness

Root-level GitHub hygiene now includes:

- `README.md` (workspace map + build instructions),
- `.gitignore` (cache/log/intermediate filtering),
- `LICENSE` (MIT),
- `CONTRIBUTING.md` (workflow and reproducibility expectations),
- this consolidated report under `report/`.

8 What a maintainer should change next

High-value next steps are:

1. move run-metric extraction to deterministic scripts (avoid manual log parsing),
2. add seed-sweep aggregation for Task 2/Task 6 to report mean±std,
3. add minimal CI checks for script importability and report build,
4. version output artifacts by run-id to prevent accidental overwrite.

9 Discussion

From a software-maintenance perspective, the strongest property of this workspace is explicit artifact discipline: paths, logs, figures, and summaries are aligned across tasks. The main weakness is that some experimental metrics are still manually summarized from logs/checkpoints rather than generated by a single deterministic reporting script. Addressing that gap would significantly reduce maintenance cost and reviewer friction.

10 Conclusion

The codebase is now structured for handoff: each task is executable and documented, rationale is explicit, and outputs are consolidated into maintainable reporting artifacts.